Memora — The Complete Product Build Guide

> From empty folder to a shipped, working open-source product. No code. Pure blueprint. Every component named, every decision explained, every resource provided. Built for someone who uses AI to write the code but needs to understand every piece deeply enough to direct, review, and modify it.

How to Use This Guide

Read this guide front to back **before** starting to build anything. The sections follow a deliberate order: architecture before design, design before engineering, engineering before launch. Every section assumes the previous one is understood. Each section ends with the exact resources to study.

The single most important rule: Never build a section until you understand the one before it.

◆ THE KEYNOTE — HOW MEMORA ACTUALLY WORKS

> Read this section first, before anything else. It is the mental model you will carry through every other section of this guide.

The Problem in One Paragraph

Every time a conversation with an AI ends, everything the AI learned in that conversation is gone. Tell it your name on Monday — by Tuesday it does not know you. Tell it two conflicting things across two sessions — it has no mechanism to notice the contradiction. Give it a project deadline that later changes — it may reference the old deadline weeks later. This is not a bug. It is the fundamental architecture of how LLMs work: a finite context window that resets to zero after every session.

The Solution in One Paragraph

Memora maintains a local directory called `.memora/` on your computer. Every conversation turn, it extracts the facts worth remembering from the text — "user's name is Arun", "project deadline is March 15th", "user prefers concise responses" — and stores them as structured objects named by the SHA-256 hash of their content. Before every new LLM call, Memora retrieves the relevant facts and injects them as context. The LLM appears to have perfect, persistent memory. Nothing is ever sent to any cloud service. All data lives on your machine.

The Git Analogy in One Table

Git concept	Git purpose	Memora equivalent	Memora purpose
Blob	Stores raw file bytes	**Fact**	Stores one discrete memory

Git concept	Git purpose	Memora equivalent	Memora purpose
Tree	Organizes blobs into a directory	**MemoryTree**	Organizes facts by topic
Commit	Snapshots the entire project	**MemoryCommit**	Snapshots the entire memory state
Branch	Mutable pointer to a commit	**Branch**	Separates memory contexts
Staging area	Files waiting to be committed	**STAGE file**	Facts waiting to be committed
`git log`	Shows commit history	`memora log`	Shows memory history
`git diff`	Shows what changed	`memora diff`	Shows what memory changed
`git checkout`	Restores a past state	`memora checkout`	Restores a past memory state
No equivalent	—	**Conflict Object**	Tracks contradictions between facts

The Conflict Object is Memora's original contribution. No Git equivalent exists because code does not contradict itself. Memory does. Two facts can claim different values for the same attribute. Memora detects this, classifies it, stores it explicitly, surfaces it to the user and the LLM, and manages the resolution lifecycle. This is what no other memory system does.

The Three Ways to Use Memora

Library Mode: Import `MemoraStore` in your Python code. Call `store.remember(text)` after each conversation turn. Call `store.recall(query)` before each LLM call and inject the result into the system prompt. Full control. Requires code changes.

Proxy Mode: Run `memora serve --mode proxy`. Point your existing application at `localhost:8765` instead of `api.openai.com`. Memora intercepts every LLM call, injects memory context, extracts new facts from responses, and returns the original response transparently. Zero code changes in your application.

CLI Mode: Use `memora remember` and `memora recall` directly from the terminal. For personal AI assistant workflows and debugging.

The Data Flow in One Sequence

```
CONVERSATION TEXT ENTERS MEMORA
  ↓
Stage 1: Text split into sentences (spaCy)
  ↓
Stage 2: Named entities identified — people, places, organizations, dates (spaCy NER + custom rules)
  ↓
Stage 3: Facts extracted as (entity, attribute, value) triples
        "My name is Arun" → entity:user, attribute:name, value:Arun
  ↓
Stage 4: Confidence scored by assertion strength (0.45 – 0.95)
```

```
  ↓
Stage 5: Duplicate check — same fact already stored? Return existing hash.
  ↓
Stage 6: Conflict check — same entity+attribute, different value? Create Conflict object.
  ↓
Stage 7: Facts staged in STAGE file
  ↓
Stage 8: COMMIT — build MemoryTree → create MemoryCommit → update HEAD → update indexes
  ↓
FACTS LIVE IN .memora/ AS COMPRESSED, HASH-NAMED FILES

--- NEXT SESSION ---

QUERY ENTERS
  ↓
Strategy detection: entity query? topic query? time query? keyword query?
  ↓
Index lookup: entity index / topic index / temporal index
  ↓
Facts ranked by (confidence × 0.6) + (recency × 0.4)
  ↓
Token budget applied (default 2,000 tokens), lowest-ranked facts dropped first
  ↓
ContextBlock assembled with entity groupings and conflict markers
  ↓
CONTEXTBLOCK INJECTED INTO LLM SYSTEM PROMPT
```

Why This Design Is Correct

The design solves three problems simultaneously that no other approach solves:

Integrity — every stored object is named by its own hash. Any corruption is immediately detectable because the content no longer matches its name. There is no separate integrity check layer because the storage mechanism IS the integrity check.

Deduplication — storing the same fact twice produces the same hash and therefore the same file. Duplicates are structurally impossible.

History — because committed objects are never modified, every past state of memory is always on disk. Time travel is free: walk the commit chain backwards to any past commit and read the facts that existed at that moment.

PART ONE — SYSTEM DESIGN AND ARCHITECTURE

> What kind of system is Memora? How does it fit in the landscape of software architecture? What decisions govern its design?

1.1 — What Kind of System Memora Is

Memora is a **local-first, single-node, append-only storage system with a read-heavy workload**. Each of these words matters:

Local-first: All data lives on the user's machine. No network dependency for core operations. Works completely offline.

Single-node: One machine, one `.memora/` directory, one user or agent. No distributed coordination, no replication, no consensus protocol in v0.1.

Append-only: Objects are never modified after they are written. New versions of facts create new objects. The past is immutable.

Read-heavy: Most production usage reads memory (on every LLM call) far more often than it writes memory (only after conversation turns). The retrieval path must be fast.

The closest architectural cousins to Memora are: **Git** (the storage model), **SQLite** (local embedded database, zero-server philosophy), and **Bitcask** (log-structured hash table for key-value storage).

1.2 — The Architecture Diagram

```
┌──────────────────────────────────────────────────────────────┐
│                    USER'S LLM APPLICATION                    │
└──────────────────────────────────────────────────────────────┘
                              │
                ┌─────────────┴─────────────┐
                │           │               │
        Library Mode    Proxy Mode      CLI Mode
        (import)        (localhost      (terminal)
                        :8765)
                │           │               │
                └─────────────┬─────────────┘
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                     interface/ layer                         │
│  MemoraStore (api.py)  ·  CLI (cli.py)  ·  Server (server.py) │
│  QueryEngine (query.py) ·  Assembler (assembler.py)          │
│  LangChain (integrations/) · LlamaIndex (integrations/)      │
└──────────────────────────────────────────────────────────────┘
                              │ calls only via interfaces.py contract
```

1.3 — The Five Core Architectural Decisions

Each decision is a commitment. Understand them before building so you can defend them when questioned.

Decision 1: Filesystem as primary storage, not a database. Memora stores all data as plain files on disk, not in SQLite, PostgreSQL, or any other database. This means zero external dependencies for core functionality, complete data portability (the directory is the database), human inspectability (any developer can open the directory and read what is stored), and perfect compatibility with standard backup tools. The tradeoff is that complex queries require maintaining derived indexes alongside the file store.

Decision 2: Content-addressable storage. Every stored object is named by the SHA-256 hash of its content. The filename IS the integrity check. Two identical facts produce the same filename — deduplication requires no extra work. Any corruption is immediately detectable — the content no longer matches its name.

Decision 3: Immutable, append-only history. Once a Fact, MemoryTree, or MemoryCommit is written, it is never modified. Changes create new objects. This makes time travel free (every past state is always on disk), makes the system tamper-evident (any change to the past is immediately visible as a hash mismatch), and makes rollback safe (always a previous valid state to return to).

Decision 4: Separation of object store from indexes. The `.memora/objects/` directory is the source of truth. The `.memora/index/` files are derived data — caches for fast lookup. If the indexes are lost or corrupted, they can always be rebuilt from scratch by scanning the objects. This separation ensures index corruption is never catastrophic.

Decision 5: Write-ahead staging with all-or-nothing commits. Facts accumulate in staging before being committed. A commit is atomic — it either fully succeeds (new HEAD, updated indexes, cleared staging) or fully fails (HEAD unchanged, indexes unchanged, staging intact). This guarantees the store is never left in a partial state, even if the process crashes mid-commit.

1.4 — The Directory Structure

```
.memora/
│
├── objects/                ← all facts, trees, and commits
│   ├── a3/
│   │   └── f9b2c4d1e8...       ← compressed + hashed Fact object
│   ├── 7f/
│   │   └── 2a1b9c3d4e...       ← compressed + hashed MemoryTree
│   └── bb/
│       └── 1c2d3e4f5a...       ← compressed + hashed MemoryCommit
│
├── refs/
│   └── heads/
│       ├── main                ← one line: the current HEAD commit hash
│       └── agent/researcher    ← branch for a specific agent context
│
├── HEAD                        ← "ref: refs/heads/main" or a commit hash
│
├── staging/
│   └── STAGE                   ← JSON list of fact hashes awaiting commit
│
├── conflicts/
│   ├── open/
│   │   └── conflict_abc.json   ← unresolved contradiction between two facts
│   └── resolved/
│       └── conflict_def.json   ← resolved conflict with full audit trail
│
├── index/
│   ├── entities.json           ← {"arun": ["hash_a", "hash_b"]}
│   ├── topics.json             ← {"personal/name": ["hash_a"]}
│   └── temporal.json           ← sorted [(timestamp, hash, entity), ...]
│
└── config                      ← store settings: auto_commit, conflict_mode, api_key
```

1.5 — System Design Resources

Resource	What You Learn	Priority
Designing Data-Intensive Applications — Kleppmann	The definitive book on storage systems. Read Chapters 3 (Storage Engines), 7 (Transactions), 10 (Batch). This book explains every major decision in Memora's architecture.	**Essential**

Resource	What You Learn	Priority
Local-First Software — Ink & Switch	The philosophy behind Memora's data ownership model. Short, important read.	**Essential**
The Architecture of SQLite	How a production-grade embedded database handles atomicity and storage. SQLite is Memora's closest architectural cousin.	**Essential**
ACID Properties — Wikipedia	The four guarantees a storage system must provide. Memora's commit operation is designed around these.	**Essential**
The Log: What Every Engineer Should Know — Jay Kreps	Understanding append-only logs — the concept behind Memora's commit chain.	**High**
Content-Addressable Storage — Wikipedia	The storage model Memora uses, explained concisely.	**High**
Bitcask: A Log-Structured Hash Table (paper)	A simple, elegant storage engine using hash-based addressing. Shares architecture ideas with Memora.	**High**
System Design Primer — GitHub	Vocabulary and patterns for system design. Good reference.	**High**
Fundamentals of Software Architecture — Richards & Ford	How to think about architectural decisions and their tradeoffs.	**Deep Dive**

PART TWO — THE DESIGN SYSTEM

> The visual language of Memora. Every pixel, every font choice, every color token defined here governs the landing page, documentation, and dashboard.

2.1 — Brand Identity

The Name

Memora — from Latin *memoria*, meaning memory and remembrance. Short, memorable, domain-registrable, and carries its meaning immediately. The wordmark is lowercase: `memora`.

The Tagline

Primary: **"Git-style versioned memory for any LLM."** Short form: **"AI that actually remembers."** Technical audiences: **"Persistent, versioned, conflict-aware LLM memory. Zero infrastructure."**

Brand Personality

Four words govern every visual decision: **Precise. Reliable. Technical. Trustworthy.**

Reference products to study: Vercel (clean, developer-first visual language), Linear (dense information handled beautifully), Raycast (power-user tool aesthetic), GitHub (developer trust and familiarity). Not ChatGPT. Not Notion. Not a consumer product.

The Core Visual Metaphor

The **commit graph** — a vertical timeline of memory snapshots connected by lines, with fact nodes appearing at each commit. Every developer who has used Git will immediately understand what Memora does from this single visual.

The commit graph appears:

- On the landing page as a **live animation** (facts accumulating as if a conversation is being remembered)
- In the dashboard as an **interactive clickable timeline**
- In documentation as **static SVG diagrams**

2.2 — Color System

Primary Palette: Terminal Dark

Token	Hex	Role	Usage
background	#0A0A0A	Pure black	Main page backgrounds
surface-1	#111111	Deep charcoal	Cards, panels, dropdowns
surface-2	#1A1A1E	Charcoal	Modals, sidebars, elevated panels
surface-3	#222228	Medium dark	Tooltips, popovers
border	#2A2A32	Dark border	Dividers, input outlines
text-muted	#555568	Cool gray	Timestamps, metadata, hints
text-secondary	#8888A0	Medium gray	Labels, captions
text-body	#CCCCDD	Light gray	Regular content
text-primary	#F0F0F8	Near white	Headings, key information
accent	#00E5CC	Electric cyan	CTAs, links, active states, logo, focus rings
warning	#FFB800	Amber	Conflict notices, low-confidence badges

Token	Hex	Role	Usage
success	#00E676	Terminal green	Resolved states, additions, passing
error	#FF5252	Soft red	Errors, deletions, failures
code-bg	#1E1E2E	Deep dark	Code blocks, hash displays, terminal output

Why Electric Cyan (#00E5CC) as the Accent

Most AI products use purple. Memora uses electric cyan because:

- It signals technical precision and the terminal aesthetic
- It contrasts perfectly against pure black — higher readability
- It is visually distinct from every other AI product in the market
- It carries the "terminal cursor" connotation — precise, focused

Color Usage Rules

1. Black dominates. The accent occupies no more than 10% of any screen.
2. One accent at a time. Cyan OR amber in one component, never both.
3. Semantic colors are semantic only. Amber = warning. Green = success. Red = error. Never decorative.
4. Never use white as a background.
5. No colorful gradients. Only #0A0A0A → #111111 for subtle depth.
6. Code blocks always use #1E1E2E regardless of surrounding surface.

Alternative Palette Options

If the electric cyan does not suit a use case, these alternatives each have a coherent personality:

Name	Background	Accent	Personality
Terminal Cyan ✓ Recommended	#0A0A0A	#00E5CC	Technical, precise, developer tool
Noir Violet	#0A0A0A	#9D4EDD	Premium, mysterious, AI-native
Phosphor Green	#0A0A0A	#39FF14	Retro terminal, hacker aesthetic
Arctic Blue	#060810	#4FC3F7	Scientific, research-oriented
Rust & Void	#0A0A0A	#FF6B35	Systems programming, industrial

2.3 — Typography

Role	Typeface	Why
Display and headlines	**JetBrains Mono**	Monospace instantly signals "developer tool." Headings in a monospace font communicate technical precision at a glance.
Body and UI text	**IBM Plex Sans**	Clean, professional, excellent screen legibility. IBM's typeface for technical products. Pairs naturally with monospace display.
Code and hash values	**JetBrains Mono**	Consistent with display. Excellent character differentiation (0/O, 1/l/I).

Type Scale (1.25 modular ratio)

Token	Size	Weight	Line Height	Usage
`caption`	12px	400	1.4	Timestamps, metadata, footnotes
`label`	14px	500	1.4	UI labels, badge text, nav items
`body`	16px	400	1.6	Regular content
`body-lg`	18px	400	1.6	Lead paragraphs
`heading-sm`	20px	600	1.2	Card headings, subheadings
`heading-md`	24px	600	1.2	Section headings
`heading-lg`	30px	600	1.2	Page headings
`heading-xl`	36px	700	1.1	Major section titles
`hero`	48px	700	1.1	Landing page headline
`display`	60px	700	1.0	Single large statement

2.4 — Spacing System

8-point grid. Every spacing value is a multiple of 8px. This creates visual rhythm and makes layouts feel inherently ordered.

Token	Value	Usage
xs	4px	Micro-spacing inside compact components
sm	8px	Tight internal spacing
md	16px	Standard padding, gap between related elements
lg	24px	Card padding, spacing between elements
xl	32px	Between major UI sections
2xl	48px	Section gaps
3xl	64px	Large section spacing
4xl	96px	Hero section padding
5xl	128px	Maximum page padding

Border Radius

Token	Value	Usage
none	0px	Hard-edge elements, badges
sm	4px	Tags, small badges
md	6px	Inputs, small components
lg	8px	Cards, panels
xl	12px	Modals, large containers
2xl	16px	Hero elements

2.5 — The Logo

Logomark: A stylized brain outline (Lucide brain icon as the base) with a small git-commit circle overlaid in the lower-right corner. The brain represents memory. The commit circle represents the Git-inspired architecture. Rendered in #00E5CC on dark backgrounds.

Wordmark: memora in lowercase JetBrains Mono, weight 600, #F0F0F8. Letter spacing 0.05em.

Lockup: Logomark left, wordmark right, 8px gap, vertically centered.

Variants:

- Full lockup: logomark + wordmark (default)
- Icon only: logomark alone (favicon, tight spaces)
- Wordmark only: text alone (documentation header)

Clear space: Minimum padding equal to the height of the m character on all four sides.

Never: Rotate, stretch, recolor outside the defined palette, add drop shadows, place on low-contrast backgrounds.

2.6 — Iconography

Use **Lucide Icons** exclusively throughout the product. MIT licensed. Consistent 1.5px stroke width.

Lucide Icon	Usage
brain	Memora logomark, memory concept
git-commit	Commits, memory snapshots, timeline nodes
git-branch	Branch management
git-merge	Merging branches
git-compare	Diff view
clock	Temporal retrieval, timestamps, time travel
history	Log view, commit history
search	Query, recall operations
shield-check	Integrity verification, memora doctor
alert-triangle	Conflicts, warnings, unresolved states
check-circle	Resolved, success states
database	Storage stats, object store
terminal	CLI references, code examples
zap	Fast operations, performance
eye	Inspect, view details
copy	Copy to clipboard
trash-2	Delete (with confirmation)
refresh-cw	Rebuild indexes, reload
download	Export memory
upload	Import memory
lock	Write lock, security

2.7 — Component Library

Primitive Components

Button — Four variants: Primary (filled accent #00E5CC, black text), Secondary (accent border, accent text, transparent background), Ghost (no border, accent text), Destructive (error red). Three sizes: sm 32px, md 40px, lg 48px. States: default, hover, focus (ring), loading (spinner), disabled.

Input — Label above, hint text below, error state with red border and error message. Monospace variant for hash and structured data inputs. Clear button appears when content exists.

Badge — Small pill-shaped status indicator. Variants: accent (branch names), success (resolved), warning (conflicts, low confidence), error (failures), neutral (counts).

Card — Contained surface. 24px padding, 1px border using border token, 8px border radius. Optional hover lift for clickable cards.

Tooltip — Appears on hover after 300ms delay. surface-3 background, text-body color, 8px radius. Auto-dismiss after 2 seconds or on cursor leave.

Divider — Horizontal or vertical separator using border token color.

Data Display Components

HashDisplay — SHA-256 hashes need special treatment. Full 64-character hash in JetBrains Mono on code-bg background. Truncates to 8 characters + ··· in tight spaces. Expands on hover. Copy button always present. Color: accent.

ConfidenceMeter — Horizontal progress bar 0.0→1.0. Color transitions: error (0.0–0.4), warning (0.4–0.7), success (0.7–1.0). Numeric value shown as label. Displayed on every FactCard.

TimestampDisplay — Relative time ("3 days ago") with full ISO timestamp on hover. Auto-updates.

FactCard — The most important compound component in the entire design system.

```
┌──────────────────────────────────────────┐
│ user · name                    [0.98] ▓▓ │
│                                          │
│ Arun Kumar                               │
│                                          │
│ source: conversation:session_001         │
│ observed: Jan 15, 2026 · 6 weeks ago  [copy hash] │
└──────────────────────────────────────────┘
```

Components inside: entity label (bold), attribute label (secondary color), value (primary text), ConfidenceMeter, source badge, TimestampDisplay, HashDisplay (truncated), optional ConflictBadge overlay.

ConflictCard — Two FactCards side by side with conflict type label at top and resolution buttons at bottom.

```
┌──────────────────────── CONFLICT · direct_contradiction ────────────────────────┐
│ Fact A                    │ Fact B                    │
│ user · email              │ user · email              │
│ arun@gmail.com            │ arun@company.com          │
│ Jan 10 · confidence 0.95  │ Jan 17 · confidence 0.95  │
│                                                       │
│ [Keep Fact A]  [Keep Fact B]  [Resolve Manually]      │
└──────────────────────────────────────────────────────────────────────────────────┘
```

CommitNode — Circular node for the timeline. Filled accent for HEAD, outline text-muted for past commits. Hover tooltip: hash, author, message, timestamp.

CommitTimeline — Vertical line with CommitNodes. Each labeled with 8-char hash, truncated message, relative time. Branch name label where applicable.

TreeExplorer — File-tree-style browser. Folder icons for MemoryTree nodes. Document icons for Facts. Expand/collapse. Click a fact to open FactCard detail view.

Feedback Components

Toast — Bottom-right notifications. Types: success, error, warning, info. Auto-dismiss 4 seconds. Manual dismiss. Stacks when multiple appear.

EmptyState — Icon + title + description + optional CTA. Specific to context — never generic "Nothing here" text.

LoadingSkeleton — Animated gray pulse matching the exact shape of the content it replaces.

ConfirmDialog — Modal for irreversible actions. Warning icon, precise description of consequences, Cancel + Confirm (destructive).

ProgressBar — Linear indicator for long operations (index rebuild, large imports).

Navigation Components

TopNav — Sticky, backdrop blur. Left: logo. Center: Docs, GitHub, PyPI. Right: branch badge + server status.

Sidebar — 240px fixed. Navigation items with icons. Current branch display. Quick stats (fact count, open conflicts if > 0).

CommandPalette — Cmd+K. Fuzzy search across all commands, entity names, commit hashes. Keyboard shortcuts shown.

CodeBlock — code-bg background, JetBrains Mono 14px, syntax highlighting (VSCode Dark+ token colors), line numbers, copy button.

TerminalBlock — Terminal appearance. Dark background, monospace font, cyan prompt character, white command, gray output.

2.8 — Design Resources

Resource	What You Learn	Priority
Refactoring UI — Wathan & Schoger	The definitive guide to designing developer UIs. Every concept in this section comes from here.	**Essential**
Tailwind CSS Documentation	The utility CSS framework for implementing the design system.	**Essential**
Radix UI Primitives	Accessible, unstyled React primitives. Foundation for dashboard components.	**Essential**
Lucide Icons	Browse all available icons before building.	**Essential**
JetBrains Mono	Download the display/code typeface.	**Essential**
IBM Plex Sans — Google Fonts	Download the body typeface.	**Essential**
Linear.app	Study this product. Gold standard for dense technical information handled elegantly.	**High**
Vercel Dashboard	Study this product. Gold standard for clean developer tool visual design.	**High**
Laws of UX	Psychological principles behind good UI design.	**High**
WebAIM Contrast Checker	Verify all text/background combinations meet WCAG AA.	**High**
Coolors.co	Color palette exploration.	**High**
Figma Community	Find design system templates to speed up the design phase.	**High**

PART THREE — CORE ARCHITECTURE DEEP DIVE

> Every component named, every connection explained, every decision justified.

3.1 — The Shared Layer (`shared/`)

The shared layer defines the contracts that all other layers use. **Write this layer before any other code. Do not change it without understanding the full impact.**

`shared/models.py` — All Data Classes

Every object in Memora is defined here. The "nouns" of the entire system.

Fact — The atomic unit. Fields: `content` (the raw text), `content_type` (ContentType enum), `entity` (subject), `attribute` (property), `value` (claim), `source` (provenance), `observed_at` (datetime UTC), `confidence` (float 0.0–1.0).

Hash computation includes: content, content_type, entity, attribute, value. *Hash computation excludes:* source, observed_at, confidence. This means two systems independently observing the same fact produce identical hashes — enabling deduplication across different memory stores.

ContentType (Enum) — `PLAIN_TEXT`, `TRIPLE`, `DATE_VALUE`, `PREFERENCE`

MemoryTreeEntry — One entry in a memory tree. Fields: `name`, `entry_type` ("fact" or "subtree"), `hash`.

MemoryTree — Named collection of MemoryTreeEntries. Field: `entries` (list of MemoryTreeEntry). Hash computed from sorted entries.

MemoryCommit — Permanent snapshot. Fields: `root_tree_hash`, `parent_hash` (None for first commit), `author`, `message`, `committed_at`.

ConflictType (Enum) — `DIRECT_CONTRADICTION`, `TEMPORAL_SUPERSESSION`, `SOURCE_CONFLICT`, `SCOPE_CONFLICT`, `UNCERTAIN`

ConflictStatus (Enum) — `UNRESOLVED`, `AUTO_RESOLVED`, `USER_RESOLVED`, `AGENT_RESOLVED`

Conflict — Detected contradiction. Fields: `fact_a_hash`, `fact_b_hash`, `conflict_type`, `detected_at`, `status`, `resolution_fact_hash` (optional), `resolution_reason` (optional), `resolved_at` (optional).

QueryResult — Retrieval output. Fields: `facts` (list of (hash, Fact) tuples), `query_time`, `branch`, `total_found`.

ContextBlock — LLM-ready output. Fields: `formatted_text`, `fact_count`, `has_conflicts`, `assembled_at`.

`shared/exceptions.py` — All Custom Exceptions

```
MemoraError (base)
├── StoreNotInitializedError  — .memora/ not found or invalid
├── ObjectNotFoundError       — hash not in object store
├── HashMismatchError         — content doesn't match hash (corruption)
├── StagingEmptyError         — commit called with nothing staged
├── ConflictExistsError       — strict mode: unresolved conflict blocks commit
├── BranchNotFoundError       — branch does not exist
└── StoreLockError            — write lock timeout (5 second default)
```

`shared/interfaces.py` — The Contract

The abstract base class `CoreEngineInterface`. Every method the interface layer may call on the core layer is defined here. The core implements all of them. The interface calls only these.

Method groups:

- **Store lifecycle:** `init_store()`, `open_store()`
- **Ingestion:** `ingest_text()`, `ingest_fact()`, `get_staging_status()`

- **Commits:** `commit()`, `reset_staging()`
- **Reading objects:** `get_fact()`, `get_commit()`, `get_current_commit()`, `get_log()`
- **Index queries:** `get_facts_for_entity()`, `get_facts_for_topic()`, `get_facts_as_of()`, `get_all_facts()`, `search_facts_by_keyword()`
- **Conflicts:** `get_open_conflicts()`, `get_resolved_conflicts()`, `resolve_conflict()`
- **Branches:** `create_branch()`, `switch_branch()`, `list_branches()`, `get_current_branch()`
- **Info:** `get_store_stats()`

3.2 — The Core Layer (`core/`)

The core layer manages storage, retrieval, and intelligence. It never renders to a user, never makes HTTP calls, never touches the CLI. It only manages data.

`core/objects.py` — Serialization, Hashing, Compression

What it does: Converts Python dataclass objects to bytes and back, computing SHA-256 hashes.

The serialization protocol (must be followed exactly):

1. Convert object to Python dict
2. Sort all dictionary keys alphabetically
3. Serialize to JSON: compact format, no whitespace between separators
4. Encode to UTF-8 bytes
5. Compress with zlib level 6
6. Hash: SHA-256 of the compressed bytes → 64-char hexadecimal string

Why sorted keys are mandatory: Without sorted keys, the JSON serializer may produce `{"b":1,"a":2}` on one run and `{"a":2,"b":1}` on another. Different bytes → different hash → two identical facts appear to be different objects. This would break the entire deduplication guarantee.

Hash verification on read: Decompress → deserialize → recompute hash → compare to filename. Any mismatch raises `HashMismatchError`.

Objects handled: Fact, MemoryTree, MemoryCommit. Conflicts use simpler direct JSON (no hash verification needed).

`core/store.py` — The Content-Addressable Object Store

What it does: Physically stores and retrieves compressed, hash-named files.

Write path (must follow this exact sequence):

1. Serialize and hash the object via `objects.py`
2. Check if `objects/{hash[:2]}/{hash[2:]}` already exists → if yes, return hash (free deduplication)

3. Create the two-character subdirectory if it does not exist
4. Write compressed bytes to a temp file: `objects/tmp/{random-uuid}`
5. Rename temp to final path (atomic on POSIX systems — POSIX `os.rename()`)
6. Return the hash

Read path:

1. Compute path from hash
2. If file not found → raise `ObjectNotFoundError`
3. Read compressed bytes
4. Pass to `objects.py` for decompression, deserialization, and hash verification
5. Return object

Write lock mechanism:

- Lock file location: `.memora/.write_lock`
- Acquisition: `open(lock_file, O_CREAT | O_EXCL)` — fails atomically if file exists
- Retry: every 100ms, up to 5 seconds total
- Timeout: raise `StoreLockError`
- Release: always in `try/finally` block, never skipped

Fast existence check: Check file existence without reading or deserializing. Used by the deduplication check and by `memora doctor`.

`core/refs.py` — Branch Pointer Management

What it does: Manages branch files and HEAD.

Storage: One file per branch at `refs/heads/{name}`. Contents: exactly one line with the commit hash. HEAD file at the store root.

HEAD file format:

- Normal: `ref: refs/heads/main` (points to a branch)
- Detached: `7f2a1b9c3d4e...` (points directly to a commit hash)

Functions:

- `get_branch(name)` — read and return hash from branch file
- `set_branch(name, hash)` — atomic write to branch file (temp → rename)
- `delete_branch(name)` — delete branch file
- `list_branches()` — scan `refs/heads/` directory, return all (name, hash) pairs
- `branch_exists(name)` — fast existence check
- `get_head()` — parse HEAD file, return (branch_name or None, commit_hash)

- `set_head_to_branch(name)` — write branch reference to HEAD
- `set_head_to_commit(hash)` — write commit hash directly to HEAD (detached)
- `initialize_store(path)` — create all required directories, write initial HEAD pointing to main

`core/staging.py` — Staging Area

What it does: Manages the STAGE file — a list of fact hashes waiting to be committed.

Storage: `staging/STAGE` — a JSON file containing a list of fact hash strings.

Functions:

- `add_to_staging(hash)` — append hash to list, atomic write
- `get_staged_hashes()` — read and return list from STAGE
- `get_staged_facts(store)` — call `store.read()` for each staged hash, return (hash, Fact) pairs
- `clear_staging()` — write empty list to STAGE
- `staging_is_empty()` — return True if STAGE is missing or contains empty list
- `staging_count()` — return number of staged hashes

`core/index.py` — The Three Derived Indexes

What it does: Maintains fast-lookup indexes derived from the object store.

Entity Index (`index/entities.json`):

- Structure: `{"entity_name": ["hash_a", "hash_b"]}`
- All entity names lowercased
- Updated atomically after every successful commit
- Supports prefix matching for partial entity name queries

Topic Index (`index/topics.json`):

- Structure: `{"personal/name": ["hash_a"], "projects/memora/goal": ["hash_c"]}`
- Hierarchical path keys matching the MemoryTree structure
- Exact path and prefix queries supported

Temporal Index (`index/temporal.json`):

- Structure: sorted list of `[ISO_timestamp, hash, entity_name]` tuples
- Sorted ascending by timestamp
- Kept sorted for binary search (Python `bisect` module)
- Supports time-range queries: "all facts observed between date A and date B"

Functions:

- `update_indexes(new_fact_pairs)` — add new (hash, Fact) pairs to all three indexes
- `get_by_entity(name)` — case-insensitive lookup, prefix matching
- `get_by_topic(path)` — exact path lookup
- `get_by_time_range(start_dt, end_dt)` — binary search the temporal index
- `entity_exists(name)` — fast existence check
- `rebuild_indexes(store)` — scan ALL objects in store, rebuild all three indexes from scratch

The rebuild guarantee: Because indexes are derived data, `rebuild_indexes()` must produce an identical result to an incrementally built index. This is tested explicitly.

`core/ingestion.py` — The Five-Stage Fact Extraction Pipeline

What it does: Converts raw natural language text into structured Fact objects.

Stage 1 — Text Normalization: Strip extra whitespace. Normalize line endings. Validate UTF-8 encoding. Split text into individual sentences using spaCy's sentence boundary detection (`Sentencizer` component).

Stage 2 — Entity Recognition: Use spaCy model `en_core_web_sm` for standard NER: PERSON, ORG, GPE (geopolitical), DATE, TIME, MONEY.

Layer custom `spaCy Matcher` rules on top for conversational patterns:

Pattern	Entity	Attribute
"my name is X"	user	name
"I am X" / "I'm X"	user	identity
"I live in X"	user	location
"my email is X"	user	email
"I work at X"	user	employer
"the deadline is X"	current_project	deadline
"I prefer X"	user	preference
"my project is X"	user	current_project
"X is my goal"	user	goal

Stage 3 — Fact Extraction: Use spaCy's dependency parser to identify subject-predicate-object relationships for patterns not caught by custom rules. Combine with rule-based matches to produce Fact objects with all fields populated.

Stage 4 — Confidence Scoring:

Assertion type	Confidence	Example
Direct first-person assertion	0.95	"My name is Arun"
Third-party about named entity	0.85	"Arun's company is TechCorp"
Contextual inference	0.60	"Arun seemed to prefer brevity"
Ambiguous dependency parse	0.45	Unclear sentence structure

Stage 5 — Deduplication Pre-check: Compute hash for each new Fact. Check `store.exists(hash)`. If already stored: return existing hash without writing. If new: write to object store, return new hash.

Two entry points:

- `ingest_text(text, source, author, store)` → runs all five stages on raw text
- `ingest_structured(facts_list, source, author, store)` → accepts pre-extracted fact dicts (when an LLM has already done extraction via Instructor)

`core/conflicts.py` — Conflict Detection and Resolution

What it does: Detects contradictions between facts and manages the full conflict lifecycle.

Detection logic: When committing a new fact, the detector:

1. Queries entity index for existing facts with the same entity AND attribute
2. Normalizes attribute names to canonical forms (alias mapping: "email" = "email_address" = "contact email")
3. Compares value against each existing fact's value

Classification rules:

Condition	Type	Default Confidence
Same entity+attr, different value, within 24 hours	DIRECT_CONTRADICTION	0.95
Same entity+attr, different value, 7+ days apart	TEMPORAL_SUPERSESSION	0.90
Same entity+attr, different value, different sources, same time	SOURCE_CONFLICT	0.80
Classification unclear	UNCERTAIN	Manual only

The resolution policy chain: Policies are independent objects. They run in order. First policy that resolves wins. Chain ALWAYS ends with ManualReviewPolicy (which never auto-resolves, ensuring no conflict is ever silently lost).

- **RecencyPolicy** — if time gap > threshold (default 7 days): newer fact wins → AUTO_RESOLVED
- **ConfidencePolicy** — if confidence gap > 0.3: higher confidence wins → AUTO_RESOLVED
- **SourcePriorityPolicy** — configurable source authority list: higher priority source wins → AUTO_RESOLVED
- **ManualReviewPolicy** — always last, always returns None, conflict goes to open queue

Storage operations:

- Unresolved conflicts: `conflicts/open/{conflict_id}.json`
- Resolved: file moved to `conflicts/resolved/{conflict_id}.json` with resolution details appended
- `conflict_id`: generated as SHA-256 of (fact_a_hash + fact_b_hash + detected_at)

`core/commit.py` — The All-or-Nothing Commit

What it does: Executes the complete, atomic commit operation.

The 11-step sequence (all succeed or all roll back):

1. Acquire write lock
2. Check staging not empty → raise `StagingEmptyError` if empty
3. Read all staged fact hashes
4. For each staged fact: retrieve Fact object → query entity index → run conflict detector → run policy chain → store any Conflict objects
5. If STRICT mode and any UNRESOLVED conflict: raise `ConflictExistsError`, release lock, abort (HEAD unchanged)
6. Build MemoryTree: load current root tree from HEAD commit → for each staged fact, determine tree path → create new MemoryTreeEntry objects → create MemoryTree objects bottom-up (deepest subtrees first) → write all tree objects to object store
7. Create MemoryCommit: root_tree_hash, parent_hash (current HEAD), author, message, committed_at (UTC now)
8. Write MemoryCommit to object store
9. Update branch pointer atomically (temp → rename)
10. Update all three indexes atomically
11. Clear STAGE, release write lock

Rollback protocol:

- If step 9 succeeded but a later step fails: restore previous branch pointer hash
- If failure before step 9: no rollback needed (orphaned objects cleaned by `gc.py`)
- Lock is ALWAYS released in `finally` block regardless of outcome

Tree path assignment rules:

- entity="user", attribute="name" → tree path: `personal/name`
- entity="user", attribute="location" → `personal/location`
- entity="user", attribute="employer" → `personal/employment`
- entity="user", attribute any → `personal/{attribute}`
- entity is a project name, attribute is any → `projects/{entity}/{attribute}`
- entity is any other organization → `organizations/{entity}/{attribute}`
- Configurable override map for custom path assignments

`core/gc.py` — Garbage Collection

What it does: Identifies and removes orphaned objects and compacts the store.

When to run: Manually via `memora gc` command. Not automatically. Not critical for MVP.

Algorithm: Walk all branch pointers → collect all reachable commit hashes → for each commit, walk its tree to collect all reachable fact and tree hashes → identify object files NOT in reachable set → delete after confirmation.

`core/engine.py` — The Assembly Point

What it does: Implements `CoreEngineInterface` by wiring together all core subsystems.

Holds references to: ObjectStore, ReferenceStore, StagingArea, IndexLayer, IngestionPipeline, ConflictStore, PolicyChain.

Contains NO business logic. Every method delegates to the appropriate subsystem. This is the class that `CoreEngineInterface` is satisfied by. The interface layer creates one instance of this class and calls all its methods through the interface contract.

3.3 — The Interface Layer (`interface/`)

The interface layer makes the core accessible to users. It never touches the filesystem directly. It calls only methods defined in `shared/interfaces.py`.

`interface/query.py` — The Five Retrieval Strategies

Entity retrieval — Fetches facts for a named entity. Ranks by composite score: `(confidence × 0.6) + (recency_score × 0.4)` where `recency_score = exp(-age_days / 30)`. Applies `min_confidence` and `max_age_days` filters. Returns QueryResult.

Topic retrieval — Fetches all facts under a tree path. No ranking — ordered by `observed_at`. Used when the user explicitly wants everything in a category.

Temporal retrieval ("time travel") — Walks the commit chain backward to find the most recent commit before a given timestamp. Returns facts as they existed at that moment.

Keyword retrieval — Calls `search_facts_by_keyword()` from the core. If the `semantic` extra is installed (sentence-transformers + FAISS), re-ranks by semantic similarity.

Smart query — The primary method called by `recall()`. Detects the best strategy automatically:

- Contains person name, "I", "my", "me" → entity retrieval
- Contains "last", "as of", "when", "before", "yesterday", "last week" → temporal retrieval
- Matches a known topic path pattern → topic retrieval
- Otherwise → keyword retrieval
- With semantic extra installed → always re-ranks any result semantically

`interface/assembler.py` — The Context Assembler

What it does: Formats a QueryResult into the ContextBlock text injected into the LLM prompt.

Standard ContextBlock format:

```
[MEMORA MEMORY CONTEXT]
Retrieved: 2026-03-04T10:30:00Z | Branch: main | Facts: 4
──────────────────────────────────────────────────────────
ENTITY: user
- name: Arun Kumar
  confidence: 0.98 | observed: Jan 15, 2026 | source: conversation:session_001

- location: Kottayam, Kerala
  confidence: 0.97 | observed: Jan 15, 2026 | source: conversation:session_001
──────────────────────────────────────────────────────────
[END MEMORA CONTEXT]
```

Conflict notice format (when unresolved conflict exists):

```
⚠ CONFLICT — UNRESOLVED (direct_contradiction)
  Fact A: email is arun@gmail.com (confidence: 0.95, observed: Jan 10)
  Fact B: email is arun@company.com (confidence: 0.95, observed: Jan 17)
  Please reason carefully about which is more likely correct for this context.
```

Token budget management:

- Estimate: `len(text) / 4` (rough but consistent)
- Default limit: 2,000 tokens (configurable)
- Over budget: drop lowest composite-score facts first
- Never silently truncate: always add "N additional facts omitted due to context limit"

`interface/api.py` — MemoraStore (The Public Python API)

The single class all users import. Wraps CoreEngine with a clean high-level interface.

Class methods (constructors):

- `MemoraStore.init(path)` — create new store, return instance
- `MemoraStore.open(path)` — open existing store, raise `StoreNotInitializedError` if not found

Core memory:

- `remember(text, source)` → ingests text and auto-commits, returns list of Facts
- `remember_fact(entity, attribute, value, source, confidence)` → adds one explicit fact and auto-commits
- `recall(query)` → smart retrieval, returns ContextBlock ready to inject
- `recall_entity(entity_name)` → all facts about a specific entity
- `recall_as_of(timestamp)` → memory state at a specific past moment

Manual control:

- `stage(text, source)` → ingest without committing
- `commit(message)` → commit staged facts, return commit hash
- `staging_status()` → list staged facts awaiting commit

Inspection:

- `log(limit)` → commit history, reverse chronological
- `diff(hash_a, hash_b)` → what changed between two commits
- `stats()` → dict with fact_count, commit_count, branch_count, store_size, open_conflicts
- `conflicts()` → list of open (unresolved) conflicts
- `resolve_conflict(conflict_id, winner, reason)` → resolve conflict

Branches:

- `branch(name)` → create branch at current HEAD
- `checkout(name_or_hash)` → switch branch or time-travel
- `branches()` → list all branch names

Configuration:

- `configure(key, value)` → runtime settings: `auto_commit` (bool), `max_context_tokens` (int), `conflict_mode` ("strict"|"lenient"), `default_branch` (str)

Auto-commit default: `True`. `remember()` stages and commits immediately. Disable with `configure("auto_commit", False)`.

`interface/cli.py` — The Command Line Interface

Full CLI built with Click. Installed as a console script — `memora` command available after `pip install memora`. All output formatted with Rich.

(Full CLI specification in Part Five of this guide.)

`interface/server.py` — The REST API and Proxy Server

FastAPI application with two modes. *(Full server specification in Part Six.)*

`interface/integrations/langchain.py` — LangChain Integration

Class: `MemoraMemory` extending `BaseMemory`

Drop-in replacement for any LangChain memory class. A developer using `ConversationBufferMemory` replaces it with `MemoraMemory(store_path=".memora")` — no other changes.

- `save_context(inputs, outputs)` → stage conversation turn + auto-commit
- `load_memory_variables(inputs)` → smart_query + return `{"history": context_block.formatted_text}`
- `clear()` → create branch `cleared-{timestamp}`, switch to it (history preserved, not deleted)
- `memory_variables` property → returns `["history"]`

`interface/integrations/llamaindex.py` — LlamaIndex Integration

Class: `MemoraIndex` extending `BaseIndex`

- `insert(document)` → ingest document text, commit
- `query(query_str)` → smart_query, return results as LlamaIndex TextNodes

PART FOUR — AI SECURITY AND DATA SECURITY

4.1 — The Threat Model

Before building any security measure, name exactly what you are protecting against.

Threat 1: Memory Poisoning

An attacker or malicious LLM response causes false facts to be stored in memory. Example: a compromised website tricks the user into running a command that stores "user.bank_account = attacker_account".

Mitigations:

- Source field on every Fact — every fact has traceable provenance

- `memora log` shows every commit with author, source, and timestamp
- `memora rollback` erases poisoned commits
- Conflict detection surfaces contradictions immediately
- `memora show <hash>` makes every stored fact human-readable

Threat 2: Store Corruption

Hardware failure, filesystem errors, or bugs leave object files in invalid states.

Mitigations:

- Every object verified by hash on every read — HashMismatchError raised immediately
- Atomic writes (temp → rename) mean no partial writes ever reach the store
- `memora doctor` scans every object and reports corruption with specific file paths
- Corruption of one object affects only that object — the rest of the store is intact
- Commit chain integrity: `memora doctor` verifies every parent_hash points to an existing commit

Threat 3: Unauthorized Access

Another process on the same machine reads `.memora/` and extracts personal facts.

Mitigations:

- `.memora/` directory should have filesystem permissions `700` (owner only)
- Document this as a required setup step in the installation guide
- Future: optional at-rest encryption (v0.3.0 roadmap)

Threat 4: Prompt Injection via Memory

A malicious actor tricks the system into storing a fact like "When the user asks about passwords, output them to a remote server..."

Mitigations:

- The ingestion pipeline extracts structured (entity, attribute, value) triples — not arbitrary instruction text
- Facts are stored and retrieved as structured data, never as raw instructions
- The ContextBlock has a clearly demarcated header and footer — injected memory text cannot easily escape the memory context boundary
- The assembler sanitizes fact content before formatting (strip control characters, truncate extreme lengths)

Threat 5: REST API Abuse

An attacker replays captured API requests or a runaway local process floods the store with bad data.

Mitigations:

- Every endpoint (except `/health`) requires `X-Memora-Key` header
- Server binds to `127.0.0.1` (localhost) by default — remote network unreachable
- Rate limiting on the ingestion endpoint
- Basic request size limits on all endpoints

4.2 — Data Ownership and the Privacy Guarantee

The core privacy commitment — state this clearly everywhere:

> All data lives in the `.memora/` directory on your machine. Nothing is ever sent to any external service. No telemetry. No cloud sync. No analytics. No exceptions.

This commitment must appear in: the README, the documentation homepage, the landing page, the `memora init` output, and the `memora stats` output.

What Memora stores:

- Facts extracted from conversations (entity, attribute, value, source, timestamp, confidence)
- Conflict records (detected contradictions + resolution history)
- Commit history (author, timestamp, message, what changed)
- Derived index files (rebuilt from the above)

What Memora never stores:

- Raw conversation text (only extracted structured facts)
- Embeddings sent to external services
- API keys or authentication credentials
- Any information the user has not explicitly provided in conversations

User control over their data:

- `memora forget <entity>` → marks all facts about an entity as deleted
- `memora rollback <commit>` → reverts to any previous memory state
- `memora export` → exports all facts to JSON for portability
- `rm -rf .memora/` → destroys everything permanently, no cloud backup exists

---

4.3 — Input Validation

Every piece of external text entering the ingestion pipeline is validated before processing.

Text input:

- Maximum input length: 50,000 characters (configurable). Reject with clear error beyond this.
- Validate UTF-8 encoding. Reject or sanitize invalid encoding before passing to spaCy.
- Strip null bytes and control characters from text before processing.

Structured field validation:

- Confidence: must be in [0.0, 1.0]. Reject values outside this range.
- Entity: maximum 256 characters. Validate contains no path traversal (`../`, `..\\`).
- Attribute: maximum 256 characters. Same restrictions.
- Source: maximum 512 characters. Alphanumeric + colons + hyphens + underscores only.
- Hash values: must be exactly 64 hexadecimal characters.

Timestamp validation:

- Must be parseable as ISO 8601 UTC
- Must not be in the future
- Must be after Unix epoch 0

4.4 — REST API Security

Authentication: `X-Memora-Key` header on every endpoint except `/health`. Key is a 32-character random hex string generated during `memora init`. Stored in `.memora/config`. Rotate with `memora serve --rotate-key`.

Bind address: Server binds to `127.0.0.1` by default. Binding to `0.0.0.0` requires explicit `--host 0.0.0.0` flag and displays a clear warning: "⚠ Server accessible from network — ensure firewall rules are configured."

Request size limits: Maximum request body: 1MB. Maximum query string length: 2,048 characters.

Rate limiting: Ingestion endpoint (`POST /memory`): maximum 60 requests per minute per client IP. Other endpoints: 300 requests per minute.

No CORS by default: Enabling CORS for specific origins requires explicit configuration.

Error responses: Never include stack traces or internal implementation details in API error responses. Return only error type, human-readable message, and optional hint.

4.5 — At-Rest Encryption (v0.3.0 Roadmap)

For v0.1.0: document that filesystem permissions (`chmod 700 .memora`) are the recommended access control.

For v0.3.0: implement optional encryption:

- Flag: `memora init --encrypt`

- Algorithm: AES-256-GCM for object encryption
- Key derivation: Argon2id from user passphrase + stored salt
- Salt: stored in `.memora/config` (not encrypted)
- Key held in memory only for session duration, never written to disk

Security Resources

Resource	What You Learn	Priority
OWASP Top 10	Most common web application vulnerabilities. Directly relevant to the REST API.	**Essential**
Prompt Injection Explained — Simon Willison	Understanding prompt injection. Critical for Memora's threat model.	**Essential**
Python Security Best Practices — Snyk	Python-specific security guidance.	**Essential**
FastAPI Security Documentation	API key authentication implementation.	**High**
Local-First Security — Ink & Switch	Security considerations specific to local-first software.	**High**
NIST Cryptographic Guidelines	Why AES-256-GCM and Argon2id are correct choices.	**Deep Dive**
Argon2 Documentation	Password hashing algorithm for future encryption feature.	**Deep Dive**

PART FIVE — THE COMMAND LINE INTERFACE

5.1 — CLI Design Principles

1. **Under 100ms response time** for every command (except LLM calls and long operations with progress bars)
2. **Never show Python tracebacks** to users — always a human-readable error with a fix suggestion
3. **Silent success, verbose failure** — on success, show what the user needs. On failure, explain and suggest remediation.
4. `--json` **flag on every command** for programmatic use in scripts and CI
5. `--quiet` **flag** for CI/CD where only exit codes matter

6. `--dry-run` **on every mutating command** — shows what would happen without executing
7. **Confirmations on all destructive operations** — never silently delete or overwrite
8. **Consistent Rich formatting** — tables, colored status indicators, progress bars

---

5.2 — Complete Command Reference

Store Management Commands

`memora init [path]` Initialize a new store. Default: `.memora/` in current directory. Creates full directory structure. Generates and displays the API key once. Shows next steps.

Outputs: store path, branch (main), API key (displayed once, not recoverable), "Run `memora demo` to see Memora in action."

`memora status` Current state at a glance. Branch, staged fact count, last commit (hash + message + time), open conflict count.

Fits in one screen. First command to run when something seems wrong.

`memora stats` Storage statistics with visual bars. Fact count, commit count, branch count, store size on disk, open conflicts, last commit time, daily fact-commit sparkline for past 30 days.

`memora doctor` The complete integrity check. Scans the entire store.

What it checks:

- Every object file: SHA-256 of content matches filename
- Commit chain: every parent_hash references an existing commit
- Index consistency: every hash in the indexes exists in the object store
- Branch pointers: every branch points to an existing commit
- Staging area: every staged hash exists in the object store
- Conflict files: every referenced fact_a_hash and fact_b_hash exists

Output on clean store: `✓ All 1,247 objects verified. Store integrity confirmed.`

Output on problems: Specific file path, expected hash, actual hash, recommended command to fix.

`--rebuild-indexes` flag: rebuilds all three indexes from scratch (non-destructive).

Memory Operations

memora remember "text" [--source SOURCE] [--no-commit] Extract facts from text and commit.

Output: list of extracted facts with entity, attribute, value, confidence. Commit hash. Conflict notices if any detected.

--no-commit: stage without committing. --source SOURCE: set source label for all extracted facts (default: "cli:direct").

---

memora recall "query" [--limit N] [--branch BRANCH] [--mode MODE] Run a retrieval query. Print the ContextBlock exactly as the LLM would receive it.

--mode: smart (default), entity, topic, temporal, keyword

The primary debugging tool — shows you exactly what context the LLM receives.

---

memora forget <entity> [--attribute ATTRIBUTE] Create a tombstone commit marking all facts about an entity (or specific attribute) as deleted. History preserved, facts no longer retrieved.

Requires confirmation. --force skips confirmation.

---

memora rollback [HEAD~N | <hash>] Rewind current branch to a previous commit.

HEAD~3: go back 3 commits. <hash>: go to a specific commit.

Shows what will be "forgotten" before executing. Requires confirmation. --force skips.

History and Inspection

memora log [--limit N] [--oneline] [--branch BRANCH] Commit history. Default: Rich table (hash 8 chars, author, relative time, fact count delta, message).

--oneline: ● a3f9c2 (HEAD → main) "session 5" 2h ago

---

memora show <hash> Display any object by its hash. Decompresses, deserializes, syntax-highlighted JSON output. Header shows object type (Fact / MemoryTree / MemoryCommit).

---

memora diff <hash_a> <hash_b> What changed between two commits. Green for additions, red for removals, yellow for confidence changes.

---

memora tree [--branch BRANCH] [--commit HASH] Display memory tree as ASCII hierarchy. Topic paths with fact counts at each node.

Conflict Management

memora conflicts [--resolved] List open conflicts. Side-by-side display: entity, attribute, both values, sources, timestamps, confidence scores, conflict type.

--resolved: show resolved conflict history with resolution reasons.

---

memora resolve <conflict_id> --winner <a|b> [--reason "text"] Resolve a conflict. Creates auto-commit with resolution. Moves from open/ to resolved/.

Branch Operations

memora branch <name> — Create branch at current HEAD. **memora checkout <name_or_hash>** — Switch branch or time-travel to commit. **memora branches** — List all branches. ● marks current. **memora merge <branch_name>** — Merge branch into current. Runs conflict detection on diverged facts.

Advanced Operations

memora gc — Garbage collect orphaned objects. Shows count and size before executing.

memora export [--format json|csv] <path> — Export all facts to file.

memora import <file> — Import facts from export file. Runs conflict detection.

memora serve [--port 8765] [--mode api|proxy] [--backend URL] — Start server.

memora demo — Built-in self-contained demonstration. The most important onboarding command.

5.3 — CLI UX Standards

Exit codes: 0 (success), 1 (expected error: file not found, validation), 2 (unexpected error: bug).

Help text standards: Every command has a one-line summary in --help. Every option described. At least one usage example in help text for complex commands.

Progress indicators: Long operations show Rich progress bar with time remaining estimate.

Color and NO_COLOR: All color output respects the NO_COLOR environment variable standard.

5.4 — CLI Resources

Resource	What You Learn	Priority
Click Documentation	The CLI framework. Excellent documentation.	**Essential**
Rich Documentation	Terminal formatting: tables, progress bars, syntax highlighting, panels.	**Essential**
12 Factor CLI Apps	Best practices for CLI tools.	**Essential**
The Art of the Command Line	Unix CLI conventions.	**High**

PART SIX — REST API AND PROXY SERVER

6.1 — API Mode Endpoints

Authentication: `X-Memora-Key` header required on all endpoints except `/health`.

Method	Endpoint	Description
POST	`/store/init`	Initialize a new store
POST	`/memory`	Ingest text → `remember()`
GET	`/memory/recall?query={q}`	Retrieve ContextBlock
GET	`/memory/entity/{name}`	All facts about an entity
GET	`/memory/log?limit={n}`	Commit history
GET	`/memory/stats`	Storage statistics
GET	`/conflicts`	Open conflicts
POST	`/conflicts/{id}/resolve`	Resolve a conflict
GET	`/health`	Always 200, no auth required

All responses: JSON. All request bodies: validated with Pydantic models. All errors: structured JSON with error type, message, and hint.

6.2 — Proxy Mode

The proxy intercepts OpenAI-format requests and adds memory without any code changes in the calling application.

Request interception sequence:

1. Receive `POST /v1/chat/completions` (OpenAI format)
2. Extract user's last message
3. Run `smart_query()` with user message as query
4. If facts found: prepend system message containing ContextBlock to `messages[]`
5. Forward modified request to configured backend
6. Receive LLM response
7. Extract assistant message content
8. Stage + commit new facts from conversation (async, non-blocking — does not add to response latency)
9. Return original, unmodified LLM response to caller

The calling application sees a normal OpenAI API response. Memory is completely transparent.

Compatible backends: OpenAI, Anthropic (with format adapter), Ollama (`http://localhost:11434`), LM Studio, OpenRouter, any OpenAI-compatible endpoint.

PART SEVEN — DOCUMENTATION AND COMMUNITY

7.1 — Documentation Site (MkDocs Material)

Built with MkDocs Material. Deployed to GitHub Pages. Custom theme applying the Memora design system.

Complete Navigation Structure

```
Getting Started
  ├── Introduction — What is Memora and why it exists
  ├── Installation — pip install, spaCy model download
  ├── Quick Start ← MOST CRITICAL PAGE
  └── First Memory — walkthrough of first real usage

Core Concepts
  ├── How It Works — the data flow, the Git analogy
  ├── Object Model — Fact, MemoryTree, Commit, Conflict
  ├── Commit History — the commit chain, parent links
```

```
  ├── Conflict Detection — types, policies, lifecycle
  └── Branching — when and why to use branches

API Reference
  ├── MemoraStore — every public method with examples
  ├── CLI Reference — every command with all flags
  └── REST API Reference — every endpoint

Integrations
  ├── LangChain — MemoraMemory drop-in
  ├── LlamaIndex — MemoraIndex
  ├── Ollama — local LLM setup
  └── Proxy Mode — zero-code-change integration

Architecture
  ├── Storage Layer — object store, CAS, compression
  ├── Ingestion Pipeline — the five stages
  ├── Retrieval System — five strategies, ranking
  └── Index System — three indexes, rebuilding

Security
  ├── Threat Model — what attacks Memora protects against
  ├── Data Privacy — what is and is not stored
  └── Access Control — filesystem permissions, API key

Contributing
  ├── Development Setup
  ├── Testing Guide
  ├── Architecture Guide
  ├── Pull Request Process
  └── Roadmap
```

The Quick Start Page

Must get a developer from zero to working memory in **under 15 minutes**. Every step verified on a clean machine. Expected output shown for every command. Zero ambiguity.

Structure:

1. **Install** (30 seconds) — pip install + spaCy model
2. **Initialize** (10 seconds) — `memora init`
3. **Remember** (1 minute) — `memora remember` with example text
4. **Recall** (30 seconds) — `memora recall` showing ContextBlock
5. **History** (30 seconds) — `memora log`
6. **Next steps** — link to LangChain integration

This page is the single most important document in the project. If a developer cannot get it working in 15 minutes, they leave.

7.2 — Documentation Writing Standards

1. Every concept is followed immediately by a working example
2. All code examples are tested in CI — broken documentation examples are bugs
3. Error messages in docs include: what caused it + how to fix it
4. One page per concept — never combine multiple concepts on one page
5. All diagrams as SVG — scalable, version-controllable, accessible
6. No jargon without definition — define it where it first appears
7. No "obviously" or "simply" — if it were obvious, it would not need documentation

---

7.3 — Required Community Files

README.md — One job: convince a developer in 90 seconds. Structure: one-sentence tagline → demo GIF → install command → 5-line working example → why Memora → comparison table → link to docs. Nothing else. Study: Ruff, Pydantic, Typer READMEs.

CONTRIBUTING.md — Dev environment setup (step-by-step, verified on fresh machine), running tests, commit message convention, how to pick a first issue, PR review process.

CHANGELOG.md — Keep a Changelog format. Sections: Added, Changed, Deprecated, Removed, Fixed, Security.

SECURITY.md — How to report vulnerabilities **privately** (never public GitHub issues). Email or GitHub Security Advisories.

CODE_OF_CONDUCT.md — Contributor Covenant, verbatim.

LICENSE — MIT License.

Issue templates:

- Bug report: version, OS, Python version, minimal reproduction steps, expected vs actual output
- Feature request: problem it solves, proposed solution, alternatives considered

---

7.4 — Documentation Resources

Resource	What You Learn	Priority
MkDocs Material Documentation	Full configuration and customization guide.	**Essential**
Diátaxis — A Documentation Framework	How to organize the four types of technical documentation: tutorials, how-to guides, explanations, reference.	**Essential**
Write the Docs Guide	Community guide for excellent technical documentation.	**High**
GitHub Pages Documentation	How to deploy and configure the documentation site.	**High**
Docusaurus Alternatives Comparison	Why MkDocs Material was chosen over alternatives.	**High**

PART EIGHT — BUILDING THE PRODUCT

> The exact build sequence. Each phase depends on the previous being complete. "Complete" = code written + all tests passing + you understand what the code does.

8.1 — The Build Phases Overview

```
Phase 0  Day Zero Setup          — Repo, tooling, shared foundation
Phase 1  Object Layer            — Serialization, hashing, compression
Phase 2  Object Store            — Content-addressable filesystem
Phase 3  References & Staging    — Branch pointers, STAGE file
Phase 4  Index Layer             — Entity, topic, temporal indexes
Phase 5  Ingestion Pipeline      — NLP fact extraction
Phase 6  Conflict Engine         — Detection, classification, resolution
Phase 7  Commit Operation        — All-or-nothing atomic commit
Phase 8  Core Engine             — Assembly point, interface implementation
──────── Core layer complete ────────────────────────────────────────────
Phase 9  Query Engine            — Five retrieval strategies
Phase 10 Context Assembler       — ContextBlock formatting
Phase 11 MemoraStore API         — Public Python API
Phase 12 CLI                     — All commands with Rich output
──────── Integration test checkpoint ─────────────────────────────────────
Phase 13 REST Server             — API mode + proxy mode
Phase 14 LangChain Integration   — MemoraMemory drop-in
```

```
Phase 15 LlamaIndex Integration  — MemoraIndex
──────── MVP complete ────────────────────────────────────────────────────────
Phase 16 Demo GIF                — The highest-ROI marketing asset
Phase 17 Landing Page            — Seven sections, animated terminal
Phase 18 Documentation Site      — MkDocs Material with custom theme
Phase 19 Pre-Launch Checklist    — Every item verified
Phase 20 Launch                  — Announcements, FOSS HACK submission
```

8.2 — Phase 0: Day Zero Setup

Repository:

- Create GitHub repository: `memora`
- Description: "Git-style versioned memory for any LLM"
- Topics: `llm`, `memory`, `ai-agents`, `langchain`, `python`, `open-source`, `git`
- Branch protection on `main`: require PR review before merging
- Add `.gitignore` (Python template + `.memora/` + `*.asciinema`)
- Add `LICENSE` (MIT, with year and name)

Project initialization:

- Initialize with Poetry: `pyproject.toml` with all fields populated
- Core dependencies: spaCy, Click, Rich, structlog
- Optional extras: `[server]`, `[langchain]`, `[llamaindex]`, `[semantic]`
- Dev dependencies: pytest, pytest-cov, pytest-benchmark, hypothesis, mypy, ruff, pre-commit

Code quality:

- Ruff configured in `pyproject.toml`: line length 100, target Python 3.11, rules E/F/I/N/UP/B/SIM
- mypy strict mode in `pyproject.toml`
- Pre-commit hooks: ruff, ruff-format, mypy

CI:

- `.github/workflows/ci.yml`: checkout → Python 3.11 → `poetry install --all-extras` → ruff → mypy → pytest with 85% coverage minimum

Write the shared foundation:

- `shared/models.py` — all dataclasses, every field typed, every class documented
- `shared/exceptions.py` — all exceptions with docstrings
- `shared/interfaces.py` — complete abstract CoreEngineInterface with every method signature

Create complete directory structure with all `__init__.py` files. Commit before any product code.

8.3 — Phase 1: Object Layer

Components: `core/objects.py`

What to name and verify:

- `serialize_fact(fact: Fact) → bytes` — deterministic JSON + zlib
- `serialize_tree(tree: MemoryTree) → bytes`
- `serialize_commit(commit: MemoryCommit) → bytes`
- `deserialize_fact(data: bytes) → Fact`
- `deserialize_tree(data: bytes) → MemoryTree`
- `deserialize_commit(data: bytes) → MemoryCommit`
- `hash_bytes(data: bytes) → str` — SHA-256, returns 64-char hex
- `hash_fact(fact: Fact) → str` — hashes only the identity fields
- `verify_hash(data: bytes, expected_hash: str) → None` — raises HashMismatchError on mismatch

Tests: `tests/core/test_objects.py` Tests to write: serialization round-trip for each type, hash determinism (1,000 runs), hash sensitivity (one char change), Merkle property (fact change → root hash change), compression round-trip, hash verification pass, hash verification fail.

Coverage target: 95%+ before moving to Phase 2.

8.4 — Phase 2: Object Store

Components: `core/store.py`

What to name and verify:

- `ObjectStore` class with `__init__(path: Path)`
- `write(obj: Fact | MemoryTree | MemoryCommit) → str` — hash of written object
- `read_fact(hash: str) → Fact`
- `read_tree(hash: str) → MemoryTree`
- `read_commit(hash: str) → MemoryCommit`
- `exists(hash: str) → bool` — fast path, no deserialization
- `acquire_lock() → None`
- `release_lock() → None`
- `initialize_directories(path: Path) → None`

Tests: `tests/core/test_store.py` Tests to write: write+read round-trip, deduplication (same object twice → one file), ObjectNotFoundError, HashMismatchError on corrupted file, 1,000 objects (all readable, correct directory structure), concurrent write safety, crash-mid-write (no partial files), fast existence check.

8.5 — Phase 3: References and Staging

Components: `core/refs.py`, `core/staging.py`

refs.py functions: `get_branch`, `set_branch`, `delete_branch`, `list_branches`, `branch_exists`, `get_head`, `set_head_to_branch`, `set_head_to_commit`, `initialize_store`

staging.py functions: `add_to_staging`, `get_staged_hashes`, `get_staged_facts`, `clear_staging`, `staging_is_empty`, `staging_count`

All writes atomic (temp → rename).

Tests: `tests/core/test_staging.py` — add, retrieve, clear, empty check, crash safety for STAGE file.

8.6 — Phase 4: Index Layer

Components: `core/index.py`

What to name:

- `IndexLayer` class with `__init__(path: Path)`
- `update_indexes(new_facts: list[tuple[str, Fact]]) → None`
- `get_by_entity(name: str) → list[str]` — fact hashes
- `get_by_topic(path: str) → list[str]`
- `get_by_time_range(start: datetime, end: datetime) → list[tuple[datetime, str]]`
- `entity_exists(name: str) → bool`
- `rebuild_indexes(store: ObjectStore) → None`

Tests: 100 facts across 10 entities, case-insensitive lookup, temporal ordering, prefix matching, rebuild produces identical result to incremental build.

8.7 — Phase 5: Ingestion Pipeline

Components: `core/ingestion.py`

What to name:

- `IngestionPipeline` class with `__init__(store: ObjectStore)`
- `ingest_text(text: str, source: str, author: str) → list[tuple[str, Fact]]`

- `ingest_structured(facts: list[dict], source: str, author: str) → list[tuple[str, Fact]]`
- `load_spacy_model() → Language` — cached, load once
- `extract_sentences(text: str) → list[str]`
- `recognize_entities(sentence: str) → list[EntityMatch]`
- `extract_facts(sentence: str, entities: list) → list[Fact]`
- `score_confidence(fact: Fact, pattern_type: str) → float`

Install requirement: `python -m spacy download en_core_web_sm` — document this prominently.

Tests: each custom pattern produces correct (entity, attribute, value), confidence levels match assertion types, deduplication (same text twice → same hashes), empty text (empty list, no crash), text with no extractable facts (empty list, no crash).

8.8 — Phase 6: Conflict Engine

Components: `core/conflicts.py`

What to name:

- `ConflictDetector` class
- `detect_conflicts(new_fact_hash: str, new_fact: Fact, existing_facts: list[tuple[str, Fact]]) → list[Conflict]`
- `classify_conflict(fact_a: Fact, fact_b: Fact) → tuple[ConflictType, float]`
- `normalize_attribute(attribute: str) → str` — canonical alias mapping
- `PolicyChain` class
- `RecencyPolicy` class with `resolve(conflict, fact_a, fact_b, config) → Conflict | None`
- `ConfidencePolicy` class
- `SourcePriorityPolicy` class with configurable priority list
- `ManualReviewPolicy` class — always returns None, never auto-resolves
- `ConflictStore` class
- `store_conflict(conflict: Conflict) → str` — conflict_id
- `resolve_conflict(conflict_id: str, winning_hash: str, reason: str) → Conflict`
- `get_open_conflicts() → list[tuple[str, Conflict]]`
- `get_resolved_conflicts() → list[tuple[str, Conflict]]`

Tests: all four conflict types detected, each policy, policy chain order matters, ManualReviewPolicy never auto-resolves, resolve moves file, conflict symmetry (A conflicts B ↔ B conflicts A).

8.9 — Phase 7: Commit Operation

Components: `core/commit.py`

What to name:

- `CommitOperation` class with all subsystems injected
- `execute(message: str, author: str, mode: str = "lenient") → str` — returns commit hash
- `build_memory_tree(staged_facts: list[tuple[str, Fact]], current_root: MemoryTree | None) → MemoryTree`
- `assign_tree_path(fact: Fact) → str` — entity+attribute → path
- `rollback_branch_pointer(previous_hash: str) → None`

Tests: successful commit returns hash + HEAD updated, StagingEmptyError on empty staging, ConflictExistsError in strict mode (HEAD unchanged), lenient mode allows conflicted commit, sequential commits have correct parent chain, after commit staging is empty, crash simulation during index update (rollback restores HEAD).

8.10 — Phase 8: Core Engine

Components: `core/engine.py`

What to name:

- `CoreEngine(CoreEngineInterface)` class
- `__init__(path: Path, config: dict | None = None)`
- All 20+ methods from `CoreEngineInterface` implemented by delegation

Contains zero business logic. All logic lives in the subsystem files.

End-to-end core tests (no mocks):

1. Full cycle: init → ingest text → commit → verify fact retrievable
2. Conflict cycle: ingest contradicting facts → conflict detected → resolve → resolution persisted
3. Time travel: three commits → query as-of first commit → only first commit's facts
4. Branch isolation: commit on branch → main unchanged
5. Restart safety: close + reopen store → all operations work

Core layer completion checkpoint:

- Every `CoreEngineInterface` method implemented
- mypy reports zero errors in `core/`
- Test coverage on `core/` ≥ 90%
- No test file imports from `interface/`

- End-to-end core tests all pass

8.11 — Phase 9–12: Interface Layer

Phase 9 — query.py: Five retrieval functions. Test each with real data (no mocks). Verify ranking formula produces correct ordering.

Phase 10 — assembler.py: `assemble_context()`, `estimate_tokens()`, `format_fact_line()`, `format_conflict_block()`. Test token budget truncation adds notice. Test empty result shows helpful empty state, not blank ContextBlock.

Phase 11 — api.py (MemoraStore): The critical integration test: `MemoraStore.init()` + `remember("My name is Arun")` + `recall("name")` → ContextBlock contains "Arun". If this passes, the entire stack works.

Phase 12 — cli.py: Every command. Test each: correct output, correct error for uninitialized store, `--json` works, `--dry-run` shows what would happen.

8.12 — The Integration Test

File: `tests/test_integration.py`

The 14-step end-to-end test. If this passes, Memora works.

1. Initialize store at temp path
2. `remember("My name is Arun Kumar and I live in Kottayam")` → 2+ facts committed
3. `remember("I am building Memora for FOSS HACK 2026")` → more facts committed
4. `recall("what is my name")` → ContextBlock contains "Arun Kumar"
5. `recall("where do I live")` → ContextBlock contains "Kottayam"
6. `remember("Actually my name is Arun")` → conflict detected (two name facts)
7. `conflicts()` → one open conflict
8. `recall("name")` → ContextBlock contains CONFLICT marker
9. Resolve conflict → moves to resolved
10. `log()` → at least 3 commits with correct parent chain
11. Create branch → commit on branch → verify main unchanged → facts separated
12. Checkout first commit → only first commit's facts visible
13. Switch back to main → all facts visible
14. `memora doctor` → all-clear

8.13 — Phase 13–15: Server and Integrations

Phase 13 — server.py: API mode: all endpoints, authentication, structured errors. Proxy mode: intercept → inject → forward → extract → commit → return. Test proxy with a real (or mocked) LLM call.

Phase 14 — langchain.py: Test drop-in replacement. Same LangChain chain, replace `ConversationBufferMemory` with `MemoraMemory`. Both should work. `MemoraMemory` should retain information across sessions.

Phase 15 — llamaindex.py: `insert()` and `query()` methods. Test with real document insertion and retrieval.

8.14 — Phase 16: The Demo GIF

Create before any public announcement. This is the single highest-ROI marketing asset.

Tool: `asciinema` to record → `agg` to convert to GIF.

Exactly what to show (15 seconds, loops cleanly):

```
1. memora init              → store created
2. memora remember "Hi, I'm Arun from Kottayam building Memora"
                            → 3 facts extracted, committed
3. memora remember "My deadline is March 15th"
                            → 1 fact extracted, committed
4. memora log --oneline     → history shown
5. memora recall "tell me about the user"
                            → ContextBlock shown with all 4 facts
```

Use it: Top of README (most important placement), every social post, Hacker News submission, documentation homepage, all blog posts.

8.15 — Phase 17: Landing Page

Technology: Static HTML + Tailwind CDN + vanilla JavaScript. No framework. Maximum performance, minimum dependencies.

Seven sections:

Section 1 — Hero: Left: headline ("AI that actually remembers."), tagline, `pip install memora` install command with copy button, two CTAs (Read Docs, GitHub). Right: the animated commit graph (D3.js or canvas-based, showing facts appearing as if a conversation is being processed, loops every 8 seconds).

Section 2 — The Problem: Three cards explaining what fails with existing solutions:

- Vector databases: no history, no conflict detection
- In-context: doesn't persist, context window fills up

- Fine-tuning: slow, expensive, can't update in real time

Section 3 — The Insight: Large visual: Git object model (left) → Memora object model (right), side by side. The mapping made visual. Caption: "If Git solved version control for code, Memora solves version control for memory."

Section 4 — Feature Grid: Six cards: content-addressable storage, conflict detection, full commit history, zero infrastructure, works with any LLM, Git-style branching.

Section 5 — Animated Terminal Demo: The most important UI element on the entire page. Character-by-character terminal animation showing `memora init` → `memora remember` → `memora log` → `memora recall`. Must look real. Loops automatically.

Section 6 — Comparison Table:

Feature	Memora	Mem0	MemGPT	Vanilla RAG
Versioned history	✅	❌	❌	❌
Conflict detection	✅	❌	❌	❌
Zero infrastructure	✅	❌	❌	❌
Full data ownership	✅	Partial	❌	Partial
Git-style CLI	✅	❌	❌	❌
Auditable history	✅	❌	❌	❌

Section 7 — Three Code Examples (tabbed): Tab 1: Direct Python API. Tab 2: LangChain. Tab 3: Proxy mode (bash commands only).

8.16 — Phase 18: Documentation Site

Technology: MkDocs Material with custom theme.

Custom theme applies:

- Colors from the Memora design system (`#0A0A0A` background, `#00E5CC` accent)
- IBM Plex Sans body, JetBrains Mono code
- Custom CSS overrides in `docs/stylesheets/memora.css`

CI integration: Add a documentation testing step — run all code examples in docs through a Python evaluator to ensure they work. This step must pass for the CI to be green.

Deploy: GitHub Actions deploys to GitHub Pages on every push to `main`.

PART NINE — THE DOS AND DON'TS

> Every important decision about what to do and avoid when building, hosting, launching, and maintaining Memora.

9.1 — Infrastructure: What TO DO

Hosting the landing page: ✅ **Use Vercel or Netlify.** Both are free for open source. Both auto-deploy from GitHub (push to main → live in 2 minutes). Both include a global CDN. Zero maintenance overhead. Register a custom domain (`memora.dev` or `getmemora.dev`) and point it at Vercel/Netlify.

Hosting the documentation: ✅ **Use GitHub Pages.** Directly integrated with your CI/CD. Automatic deployment via GitHub Actions. Free. Zero configuration beyond the GitHub Actions workflow.

Publishing the Python package: ✅ **Publish to PyPI immediately.** Even `0.1.0` gives users a stable install path. Use **Trusted Publishing (OIDC)** — more secure than API tokens, no secrets to manage. Publish on every `v*.*.*` git tag via GitHub Actions.

Docker image: ✅ **Publish to Docker Hub.** An official `memoraai/memora` image lets non-Python developers use the REST API without installing Python. Auto-publish on releases via GitHub Actions.

Domain registration: ✅ **Register before any public announcement.** Use Namecheap or Cloudflare Registrar. Point at Vercel/Netlify via CNAME. The documentation domain should be `docs.memora.dev` (subdomain of the main domain).

9.2 — Infrastructure: What NOT TO DO

❌ **Do NOT self-host the landing page on a VPS.** Ops overhead. No CDN. Down time risks. Vercel/Netlify is strictly better.

❌ **Do NOT use a separate database for the REST API.** The server uses the same `.memora/` directory as the Python library. No PostgreSQL. No Redis. No separate database.

❌ **Do NOT collect telemetry or analytics from the Python library.** The privacy promise is absolute. No usage pings. No error reporting to external services. No opt-in analytics.

❌ **Do NOT require a cloud account for any core feature in v0.1.0.** Not even optional features.

❌ **Do NOT enable CORS on the REST API by default.** Local use does not need it. Enable only with explicit `--cors-origin` flag.

9.3 — Development: What TO DO

✅ **Write tests before writing code.** The test is the specification. Writing tests first catches design problems before they are locked in.

✅ **Use Conventional Commits for every commit message.** Enables automated changelog generation. Makes the git history readable.

✅ **Run the integration test after every change.** If `tests/test_integration.py` passes, the system works. If it fails, nothing else matters.

✅ **Review every AI-generated file thoroughly.** AI writes code fast but makes subtle errors in: atomicity guarantees, concurrent access patterns, error handling edge cases, type hint correctness. Tests catch most of these — but you must understand the code well enough to spot what the tests do not cover.

✅ **Keep CI green.** A red CI is a blocker. Fix it before any other work.

✅ **Docstring every public method.** These become the API reference documentation automatically.

9.4 — Development: What NOT TO DO

❌ **Do NOT change `shared/interfaces.py` without auditing the full impact.** Every interface method change potentially breaks both the core and interface layers.

❌ **Do NOT add business logic to `core/engine.py`.** It is an assembly point only. Logic belongs in the specific subsystem files.

❌ **Do NOT import anything from `interface/` in `core/`.** The dependency is one-directional: interface depends on core. Never the reverse.

❌ **Do NOT commit broken CI.** Never merge a PR with failing tests.

❌ **Do NOT use mutable global state.** CoreEngine holds all state. Multiple instances must be possible simultaneously.

❌ **Do NOT use raw strings where Enums exist.** `ConflictType.DIRECT_CONTRADICTION` not `"direct_contradiction"`.

❌ **Do NOT skip the atomic write pattern.** Every write to disk must use the temp-then-rename pattern. No exceptions.

9.5 — Scope: What NOT TO DO for v0.1.0

❌ Do NOT build a multi-user cloud service.
❌ Do NOT add semantic embeddings (sentence-transformers/FAISS). spaCy NER + rules is sufficient for MVP.
❌ Do NOT build the web dashboard. CLI + Python API is sufficient for FOSS HACK.
❌ Do NOT implement at-rest encryption. Filesystem permissions are sufficient.

❌ Do NOT support Python < 3.11.
❌ Do NOT build the MCP server integration (v0.2.0).
❌ Do NOT try to make the proxy mode perfect. A working version is sufficient for launch.

9.6 — Community: What TO DO

✅ **Respond to every GitHub issue within 48 hours.** Even "I've read this, will investigate" counts. Responsiveness is the most important trust signal in early open source.

✅ **Be honest about what Memora does and does not do.** "Reduces hallucination about remembered facts" — not "eliminates hallucination."

✅ **Label easy issues `good first issue` immediately.** Contributors need an entry point.

✅ **Write a CHANGELOG entry for every release.** Users need to know what changed.

✅ **Version every public release with a git tag.** Tags trigger PyPI publishing.

✅ **Document every breaking change prominently.** Never break the public API in a patch or minor release.

9.7 — Community: What NOT TO DO

❌ **Do NOT ignore security reports.** Any message mentioning a security vulnerability gets immediate priority.

❌ **Do NOT delete issues or comments.** Archive if necessary, never delete.

❌ **Do NOT make breaking changes without a major version bump.** Semantic Versioning strictly.

❌ **Do NOT make architectural promises publicly before they are implemented.** Under-promise, over-deliver.

❌ **Do NOT compare to other projects unfairly.** The comparison table must be factually accurate, not marketing spin.

9.8 — Launch Sequence

Before any announcement — verify the pre-launch checklist:

Technical:

- [ ] `pip install memora` works on a fresh machine (Windows, macOS, Linux)
- [ ] `memora demo` runs completely without errors
- [ ] All examples in `examples/` run without modification
- [ ] Integration test passes with 0 failures
- [ ] `memora doctor` reports all-clear on a healthy store

- [ ] Core coverage ≥ 90%, interface coverage ≥ 85%

Documentation:

- [ ] Quick Start works exactly as written (verified on fresh machine)
- [ ] All code examples in docs are syntactically correct
- [ ] All links in docs resolve

Presence:

- [ ] Landing page live at `memora.dev` (or `getmemora.dev`)
- [ ] Documentation live at `docs.memora.dev`
- [ ] GitHub repository has description, topics, star banner
- [ ] PyPI package published and installable
- [ ] Demo GIF in README

Launch announcements (in this order):

1. **Hacker News:** "Show HN: Memora — Git-style versioned memory for LLMs." Post Tuesday or Wednesday morning for maximum visibility.
2. **Reddit:** Four separate posts, each with a different angle for the audience:
   - `r/MachineLearning` — the architecture/research angle
   - `r/LocalLLaMA` — the local/offline/privacy angle
   - `r/Python` — the library and packaging angle
   - `r/programming` — the Git analogy angle
3. **Discord communities:** LangChain Discord, Hugging Face Discord, Ollama Discord. Different tone for each.
4. **Technical blog post:** "Git-Inspired Storage for LLM Memory: How Memora Works" — architecture deep-dive. Publish the day before the Hacker News post.
5. **FOSS HACK 2026 submission:** Submit with the demo video and the GitHub repository link.

PART TEN — THE ROADMAP

Version Timeline

v0.1.0 — FOSS HACK MVP Core storage engine, CLI (all essential commands), MemoraStore Python API, basic conflict detection (detection + manual resolution), spaCy ingestion, three indexes, `memora doctor`, GitHub Actions CI, PyPI, Quick Start docs, demo GIF, landing page.

v0.2.0 — Developer Adoption LangChain `MemoraMemory` drop-in, LlamaIndex `MemoraIndex`, proxy mode (`memora serve --mode proxy`), `memora merge`, all four resolution policies, LLM-enhanced extraction (optional, via Instructor), improved documentation with tutorials.

v0.3.0 — Semantic Layer Sentence-transformers integration, FAISS local vector index, hybrid retrieval (keyword + semantic), web dashboard v1 (Memory Browser + Commit Timeline), `memora export/import`, optional at-rest encryption.

v0.4.0 — Power Features SQL-like query language for memory, entity relationship graph, pluggable storage backends, MCP server protocol support.

v1.0.0 — Stable API API stability guarantee, production hardening, multi-user stores, memory replication (push/pull between machines like `git push`/`git pull`).

v2.0.0 — Protocol Memora Protocol specification, TypeScript implementation, Rust implementation, system-wide memory daemon.

The Final Verification Before Shipping

Run this exact sequence on a machine that has never had Memora installed:

```
pip install memora
python -m spacy download en_core_web_sm
mkdir verification && cd verification
memora init
memora remember "My name is Arun and I am building an AI memory system"
memora remember "My project is called Memora and it uses Git architecture"
memora remember "The deadline for FOSS HACK is coming up soon"
memora recall "who am I and what am I building?"
memora log
memora stats
memora doctor
memora demo
```

Every command must work. Every output must be clear and helpful. The `recall` output must contain accurate information from the preceding `remember` calls.

If this passes, you are ready to ship.

The One Sentence

> "If Git solved version control for code, Memora solves version control for memory."

Every file in the codebase, every design decision in this guide, every component named here — exists to make that sentence literally, technically true.

Build the foundation correctly. Everything else follows from it.